



TECHNICAL WHITEPAPER

# Digital Design Experiments on Crucible

Igny CLI · Ignition IO

## ABSTRACT

## Overview

This whitepaper documents five foundational Verilog experiments executed on the Ignition platform — a purpose-built environment for digital hardware design, simulation, and verification. The experiments span sequential logic, combinational circuits, and finite-state machines, providing a structured walkthrough of classical digital design problems solved through RTL design and simulation.

Each experiment runs through the **Ignity CLI**, the command-line interface to the **Crucible** reproducible-tooling workbench. Crucible provisions several open source EDA tools across the entire chip design pipeline. For these experiments Icarus Verilog & GTKWave— no global tool installation required.

Every experiment follows a standard academic lab structure — Aim, Apparatus, Theory, RTL Design, Procedure, Observations, and Result — making this document suitable as both a product demonstration and onboarding material for engineers and evaluators.

## CONTENTS

## Table of Contents

|    |                                                |    |
|----|------------------------------------------------|----|
| 1. | Introduction to the Ignition Platform          | 3  |
| 2. | Lab Setup: Getting Started with the Ignity CLI | 4  |
| —  | Experiment 01 · D Flip-Flop                    | 5  |
| —  | Experiment 02 · Controlled Up/Down Counter     | 6  |
| —  | Experiment 03 · Seven-Segment Decoder          | 7  |
| —  | Experiment 04 · Mealy "101" Sequence Detector  | 8  |
| —  | Experiment 05 · Three-Number Adder             | 9  |
| 3. | Summary of Results & Conclusion                | 10 |

## SECTION 1

# Introduction to the Ignition Platform

## PLATFORM OVERVIEW

Ignition is a silicon development platform for engineering teams working at the intersection of hardware design, simulation, and verification. It provides the infrastructure layer for digital design workflows — handling tool provisioning, environment isolation, and execution reproducibility — so engineers can focus on design rather than toolchain management.

## THE CRUCIBLE WORKBENCH

At the core of the platform is the **Crucible Workbench** — a managed-tooling environment that abstracts EDA tool installation, versioning, and runtime execution behind a clean, consistent interface. It binds specific tool versions to named project environments, ensuring every team member runs an identical, verified toolchain. Lock the toolset with `ign workspace lock` and reproduce it with `ign workspace sync`.

## THE IGNY CLI

The **Ign CLI** (`ign`) is the primary interface to Crucible. Engineers use `ign env create` to define isolated environments, `ign tool install` to populate them, and `ign run <tool>` to execute without requiring tools on the system PATH — eliminating environment drift and version conflicts.

## ABOUT THIS WHITEPAPER

The five experiments documented here exercise the platform's EDA simulation capability end-to-end, spanning core categories of digital logic design — from storage elements (D flip-flop) to combinational decoders (seven-segment) and finite-state machines (Mealy sequence detector). Each demonstrates the full Ignition workflow: compile and simulate through Ign, inspect waveforms in GTKWave, and verify via testbenches.

## TOOLCHAIN USED IN THIS LAB

Simulator

Icarus Verilog 14.0

Waveform Viewer

GTKWave

Interface

Ign CLI

## SECTION 2

# Quick Setup through Igny CLI

All five experiments share a single Crucible environment with the Verilog toolchain installed. Run this setup once; every experiment below reuses it.

## Step 1: Environment & Workspace Creation

```
ign y env create veril og-lab
ign y workspace create --path ./ign ytion-demo-experiments --env veril og-lab
cd ./ign ytion-demo-experiments
```

## Step 2: Tool Installation

```
ign y tool install iverilog
ign y tool install gtkwave
ign y workspace lock
```

## Real Setup Output

```
C:\Users\ignyt\Desktop >> ign y env create veril og-lab
Environment 'verilog-lab' created (stack=default)
C:\Users\ignyt\Desktop >> ign y workspace create --path ./ign ytion-demo-experiments --env veril og-lab
Workspace 'ign ytion-demo-experiments' created (env=verilog-lab)
C:\Users\ignyt\Desktop >> cd ./ign ytion-demo-experiments\
C:\Users\ignyt\Desktop\ign ytion-demo-experiments >> ign y tool install iverilog
✓ Installed iverilog@12.0.0
Source: artifact
Path: C:\Users\ignyt\Crucible\tools\iverilog\12.0.0
Bound to env: veril og-lab
C:\Users\ignyt\Desktop\ign ytion-demo-experiments >> ign y tool install gtkwave
✓ Installed gtkwave@3.3.120
Source: artifact
Path: C:\Users\ignyt\Crucible\tools\gtkwave\3.3.120
Bound to env: veril og-lab
C:\Users\ignyt\Desktop\ign ytion-demo-experiments >> |
```

### WHY IGNY?

`ign y tool install iverilog` downloads the simulator once into a shared machine inventory and *binds* it to the `verilog-lab` environment. A teammate can reproduce the exact toolset with `git clone ... && ign y workspace sync`. Nothing is installed globally.

### THE THREE COMMANDS EVERY EXPERIMENT USES

| STEP              | COMMAND                                                     | WHAT IT DOES                                                           |
|-------------------|-------------------------------------------------------------|------------------------------------------------------------------------|
| <b>Compile</b>    | <code>ign y run iverilog -o &lt;name&gt;_sim.vvp ...</code> | Elaborates design + testbench into a runnable <code>.vvp</code>        |
| <b>Simulate</b>   | <code>ign y run vvp &lt;name&gt;_sim.vvp</code>             | Runs simulation; prints log and writes <code>.vcd</code> waveform dump |
| <b>View waves</b> | <code>ign y run gtkwave &lt;name&gt;_tb.vcd</code>          | Opens the waveform dump in GTKWave for visual inspection               |

**Pass criteria:** A **PASS** is reported when a testbench prints its `... test passed.` line and calls `$finish` with no preceding `ERROR:` line. Each Verilog testbench halts on the first mismatch.

## EXPERIMENT 01

# D Flip-Flop

Sequential Logic

testbench · PASS/FAIL

## AIM

Design and simulate a positive-edge-triggered D flip-flop with a synchronous active-high reset and complementary output.

Verify its behaviour through simulation.

## PORTS

| PORT   | DIR | DESCRIPTION                    |
|--------|-----|--------------------------------|
| d_in   | in  | Data input                     |
| clock  | in  | Clock                          |
| reset  | in  | Synchronous, active-high reset |
| Q_out  | out | Stored bit                     |
| Qb_out | out | Complement of Q_out            |

## THEORY

A D (Data) flip-flop is the fundamental 1-bit storage element of sequential logic. On every rising clock edge it captures the value at `d_in` and holds it at `Q_out`, while `Qb_out` is always its logical complement. The reset is **synchronous and active-high**: it forces `Q_out = 0` only on a clock edge, never asynchronously.

## RTL DESIGN · DFF.V

```
module dff (  
    input wire d_in,  
    input wire clock,  
    input wire reset,  
    output reg Q_out,  
    output reg Qb_out  
);  
    always @(posedge clock) begin  
        if (reset) begin  
            Q_out <= 1'b0;  
            Qb_out <= 1'b1;  
        end  
        else begin  
            Q_out <= d_in;  
            Qb_out <= ~d_in;  
        end  
    end  
endmodule
```

**EXPERIMENT 01 · D FLIP-FLOP — EXECUTION & RESULTS****TEST STRUCTURE**

```
task check_outputs(input expected_q, input expected_qb);
    @(posedge clock); #1;
    if (Q_out != expected_q || Qb_out != expected_qb) begin
        $display("ERROR: got Q_out=%b Qb_out=%b", Q_out, Qb_out);
    $finish;
    end
endtask

// Test stimulus
reset = 1'b1; check_outputs(1'b0, 1'b1);
reset = 1'b0;
@(negedge clock); d_in = 1'b1; check_outputs(1'b1, 1'b0);
@(negedge clock); d_in = 1'b0; check_outputs(1'b0, 1'b1);
@(negedge clock); d_in = 1'b1; check_outputs(1'b1, 1'b0);
```

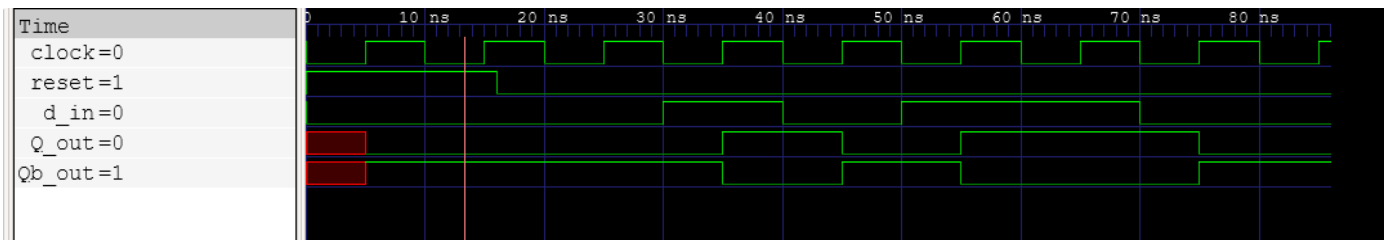
**PROCEDURE**

```
cd 01_dff
ign run iverilog -o dff_sim.vvp -s dff_tb dff.v dff_tb.v
ign run vvp dff_sim.vvp
ign run gtkwave dff_tb.vcd
```

**CRUCIBLE EXECUTION OUTPUT**

```
C:\Users\ignyt\Desktop\ignytion_demo_experiments\01_dff >> ign run iverilog -o dff_sim.vvp -s dff_tb dff.v dff_tb.v
C:\Users\ignyt\Desktop\ignytion_demo_experiments\01_dff >> ign run vvp dff_sim.vvp
VCD info: dumpfile dff_tb.vcd opened for output.
Time=0 | reset=1 | d_in=0 | Q_out=x | Qb_out=x
Time=5000 | reset=1 | d_in=0 | Q_out=0 | Qb_out=1
Time=16000 | reset=0 | d_in=0 | Q_out=0 | Qb_out=1
Time=30000 | reset=0 | d_in=1 | Q_out=0 | Qb_out=1
Time=35000 | reset=0 | d_in=1 | Q_out=1 | Qb_out=0
Time=40000 | reset=0 | d_in=0 | Q_out=1 | Qb_out=0
Time=45000 | reset=0 | d_in=0 | Q_out=0 | Qb_out=1
Time=50000 | reset=0 | d_in=1 | Q_out=0 | Qb_out=1
Time=55000 | reset=0 | d_in=1 | Q_out=1 | Qb_out=0
Time=70000 | reset=0 | d_in=0 | Q_out=1 | Qb_out=0
Time=75000 | reset=0 | d_in=0 | Q_out=0 | Qb_out=1
DFF test passed.
dff_tb.v:59: $finish called at 86000 (1ps)
C:\Users\ignyt\Desktop\ignytion_demo_experiments\01_dff >> ign run gtkwave dff_tb.vcd
```

## GTKWAVE — WAVEFORM OUTPUT



## RESULT

`Q_out` follows `d_in` delayed by one clock; `Qb_out` is always its complement. Both resolve from `x` only after the first clocked reset — confirming synchronous behaviour. **DFF test passed.**

## EXPERIMENT 02

# Controlled Up/Down Counter

Sequential FSM

testbench · PASS/FAIL

## AIM

Design and verify a 4-bit counter whose counting behaviour is selected at run time by a control input `w`.

## PORTS

| PORT       | DIR | DESCRIPTION                      |
|------------|-----|----------------------------------|
| clock      | in  | Clock                            |
| reset      | in  | Synchronous reset                |
| w          | in  | Mode: 1 = up-by-2, 0 = down-by-1 |
| count[3:0] | out | 4-bit count output               |

## THEORY

A controlled counter changes its update rule based on a mode input. When `w = 1` (up-by-2 with wrap): if `count ≥ 8`, subtract 8; otherwise add 2 — producing the even sequence 0, 2, 4, 6, 8, 0, ... When `w = 0` (down-by-1): decrement, wrapping 0 → 15. A synchronous active-high reset clears `count` to 0.

## RTL DESIGN · CONTROLLED\_COUNTER.V

```

module controlled_counter (
    input wire    clock,
    input wire    reset,
    input wire    w,
    output reg [3:0] count
);
    always @(posedge clock) begin
        if (reset)
            count <= 4'd0;
        else if (w) begin
            if (count >= 4'd8)
                count <= count - 4'd8;
            else
                count <= count + 4'd2;
        end
        else
            count <= count - 4'd1;
        end
    end
endmodule

```

**EXPERIMENT 02 · CONTROLLED COUNTER – EXECUTION & RESULTS****TEST STRUCTURE**

```
task check_count(input [3:0] expected);
    @(posedge clock); #1;
    if (count !== expected) begin
        $display("ERROR: w=%b expected=%0d got=%0d", w, expected, count);
        $finish;
    end
endtask

// Test stimulus
reset = 1'b1; check_count(4'd0);
reset = 1'b0;
w = 1'b1; // Count up by 2
check_count(4'd2);
check_count(4'd4);
check_count(4'd6);
check_count(4'd8);
check_count(4'd0); // Wrap at 10
w = 1'b0; // Count down by 1
check_count(4'd15);
check_count(4'd14);
check_count(4'd13);
```

**PROCEDURE**

```
cd 02_controlled_counter

igny run iverilog -o counter_sim.vvp -s controlled_counter_tb controlled_counter.v controlled_counter_tb.v

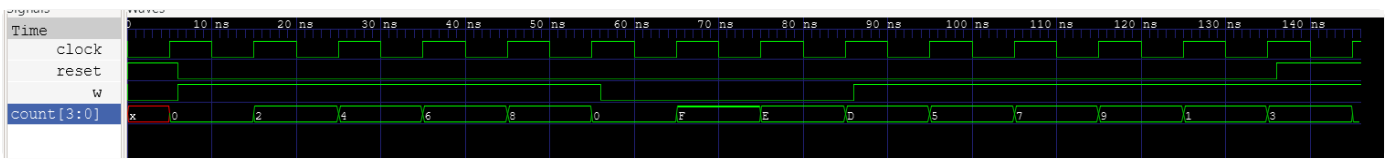
igny run vvp counter_sim.vvp

igny run gtkwave controlled_counter_tb.vcd
```

**CRUCIBLE EXECUTION OUTPUT**

```
C:\Users\ignyt\Desktop\ignytion_demo_experiments\02_controlled_counter >> igny run iverilog -o controlled_counter.vvp -s controlled_counter_tb c
ontrolled_counter.v controlled_counter_tb.v
C:\Users\ignyt\Desktop\ignytion_demo_experiments\02_controlled_counter >> igny run vvp controlled_counter.vvp
VCD info: dumpfile controlled_counter_tb.vcd opened for output.
Time=0 | reset=1 | w=0 | count=x
Time=5000 | reset=1 | w=0 | count=0
Time=6000 | reset=0 | w=1 | count=0
Time=15000 | reset=0 | w=1 | count=2
Time=25000 | reset=0 | w=1 | count=4
Time=35000 | reset=0 | w=1 | count=6
Time=45000 | reset=0 | w=1 | count=8
Time=55000 | reset=0 | w=1 | count=0
Time=60000 | reset=0 | w=0 | count=0
Time=65000 | reset=0 | w=0 | count=15
Time=75000 | reset=0 | w=0 | count=14
Time=85000 | reset=0 | w=0 | count=13
Time=86000 | reset=0 | w=1 | count=13
Time=95000 | reset=0 | w=1 | count=5
Time=105000 | reset=0 | w=1 | count=7
Time=115000 | reset=0 | w=1 | count=9
Time=125000 | reset=0 | w=1 | count=1
Time=135000 | reset=0 | w=1 | count=3
Time=136000 | reset=1 | w=1 | count=3
Time=145000 | reset=1 | w=1 | count=0
Controlled counter test passed.
controlled_counter_tb.v:69: $finish called at 146000 (1ps)
C:\Users\ignyt\Desktop\ignytion_demo_experiments\02_controlled_counter >> igny run gtkwave controlled_counter_tb.vcd
```

## GTKWAVE — WAVEFORM OUTPUT



## RESULT

Counter steps +2 (with wrap at 8) when `w=1` and -1 when `w=0`; resets cleanly to 0. **Controlled counter test passed.**

## EXPERIMENT 03

# Seven-Segment Decoder

Combinational Logic

testbench · PASS/FAIL

## AIM

Design and verify a combinational decoder that converts a 4-bit hexadecimal input into the seven-segment display pattern for digits 0–F.

## THEORY

A seven-segment display has seven segments labelled a–g. This decoder is purely combinational — each of the 16 input values maps to a fixed 7-bit active-high pattern ordered {a, b, c, d, e, f, g}. For example, 0 → 1111110 lights every segment except g.

## PORTS

| PORT         | DIR | DESCRIPTION                 |
|--------------|-----|-----------------------------|
| hex_in[3:0]  | in  | Hex digit 0–F               |
| seg_out[6:0] | out | Active-high segments {a..g} |

## RTL DESIGN · SEVEN\_SEGMENT\_DECODER.V

```

module seven_segment_decoder (
    input wire [3:0] hex_in,
    output reg [6:0] seg_out
);
    always @(*) begin
        case (hex_in)
            4'h0: seg_out = 7'b1111110; 4'h8: seg_out = 7'b1111111;
            4'h1: seg_out = 7'b0110000; 4'h9: seg_out = 7'b1111011;
            4'h2: seg_out = 7'b1101101; 4'hA: seg_out = 7'b1110111;
            4'h3: seg_out = 7'b1111001; 4'hB: seg_out = 7'b0011111;
            4'h4: seg_out = 7'b0110011; 4'hC: seg_out = 7'b1001110;
            4'h5: seg_out = 7'b1011011; 4'hD: seg_out = 7'b0111101;
            4'h6: seg_out = 7'b1011111; 4'hE: seg_out = 7'b1001111;
            4'h7: seg_out = 7'b1110000; 4'hF: seg_out = 7'b1000111;
            default: seg_out = 7'b0000000;
        endcase
    end
endmodule

```

## EXPERIMENT 03 · SEVEN-SEGMENT DECODER — EXECUTION & RESULTS

### TEST STRUCTURE

```
function [6:0] expected_seg(input [3:0] value);
  case (value)
    4'h0: expected_seg = 7'b1111110;
    4'h1: expected_seg = 7'b0110000;
    4'h2: expected_seg = 7'b1101101;
    ...
    4'hF: expected_seg = 7'b1000111;
    default: expected_seg = 7'b0000000;
  endcase
endfunction

// Test stimulus: sweep all 16 hex values
for (i = 0; i < 16; i = i + 1) begin
  hex_in = i[3:0];
  #10;
  if (seg_out != expected_seg(hex_in)) begin
    $display("ERROR: hex_in=%h expected=%b got=%b",
      hex_in, expected_seg(hex_in), seg_out);
    $finish;
  end
end
end
```

### PROCEDURE

```
cd 03_seven_segment_decoder

igny run iverilog -o seg_sim.vvp -s seven_segment_decoder_tb seven_segment_decoder.v seven_segment_decoder_tb.v

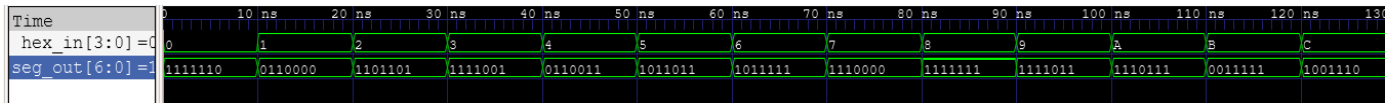
igny run vvp seg_sim.vvp

igny run gtkwave seven_segment_decoder_tb.vcd
```

### CRUCIBLE EXECUTION OUTPUT

```
C:\Users\ignyt\Desktop\ignytion_demo_experiments\03_seven_segment_decoder >> igny run vvp seven_segment_decoder.vvp
VCD info: dumpfile seven_segment_decoder_tb.vcd opened for output.
Time=0 | hex_in=0 | seg_out=1111110
Time=10000 | hex_in=1 | seg_out=0110000
Time=20000 | hex_in=2 | seg_out=1101101
Time=30000 | hex_in=3 | seg_out=1111001
Time=40000 | hex_in=4 | seg_out=0110011
Time=50000 | hex_in=5 | seg_out=1011011
Time=60000 | hex_in=6 | seg_out=1011111
Time=70000 | hex_in=7 | seg_out=1110000
Time=80000 | hex_in=8 | seg_out=1111111
Time=90000 | hex_in=9 | seg_out=1111011
Time=100000 | hex_in=a | seg_out=1110111
Time=110000 | hex_in=b | seg_out=0011111
Time=120000 | hex_in=c | seg_out=1001110
Time=130000 | hex_in=d | seg_out=0111101
Time=140000 | hex_in=e | seg_out=1001111
Time=150000 | hex_in=f | seg_out=1000111
Seven-segment decoder test passed.
seven_segment_decoder_tb.v:56: $finish called at 160000 (1ps)
C:\Users\ignyt\Desktop\ignytion_demo_experiments\03_seven_segment_decoder >> igny run gtkwave seven_segment_decoder_tb.vcd
```

## GTKWAVE — WAVEFORM OUTPUT



## RESULT

All 16 hex inputs (0–F) produce their correct active-high segment patterns. The testbench sweeps all values and verifies against an independent reference function. **Seven-segment decoder test passed.**

## EXPERIMENT 04

# Mealy "101" Sequence Detector

Sequential FSM

testbench · Overlap supported

## AIM

Design and verify a Mealy FSM that detects the pattern **101** in a serial bit stream, with overlapping detection allowed.

## THEORY

In a Mealy machine, output depends on both the current state *and* the current input — so detection is signalled in the **same cycle** the final bit arrives. The FSM uses three states: S0 (idle), S1 (saw 1), S2 (saw 10). Overlap is supported: the trailing 1 of one match is the leading 1 of the next.

## RTL DESIGN · MEALY\_SEQ\_DET.V

```
module mealy_seq_det(
    input wire clock, reset, seq_in,
    output wire det_o
);
    parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10;
    reg [1:0] present_state, next_state;

    always @(posedge clock) begin
        if (reset) present_state <= S0;
        else present_state <= next_state;
    end

    always @(*) begin
        case (present_state)
            S0: next_state = seq_in ? S1 : S0;
            S1: next_state = seq_in ? S2 : S0;
            S2: next_state = seq_in ? S0 : S0;
            default: next_state = S0;
        endcase
    end

    assign det_o = (present_state == S2) && seq_in;
endmodule
```

## PORTS

| PORT   | DIR | DESCRIPTION             |
|--------|-----|-------------------------|
| clock  | in  | Clock                   |
| reset  | in  | Synchronous reset → S0  |
| seq_in | in  | Serial input bit        |
| det_o  | out | High when 101 completes |

## EXPERIMENT 04 · CONTINUED

### TEST STRUCTURE

```
task drive_bit(input bit_value);
    @(negedge clock);
    seq_in = bit_value;
    history = {history[1:0], bit_value};
    expected_det = (history == 3'b101);
    #1;
    if (det_o != expected_det) begin
        $display("ERROR: bit=%b history=%b expected=%b got=%b",
            bit_value, history, expected_det, det_o);
    $finish;
    end
endtask

// Test stimulus: stream 16 bits looking for "101"
stream = 16'b1011010110010101;
apply_reset;
for (i = 15; i >= 0; i = i - 1)
    drive_bit(stream[i]);
```

### PROCEDURE

```
cd 04_mealy_seq_det
ign run iverilog -o mealy_sim.vvp -s mealy_seq_det_tb mealy_seq_det.v mealy_seq_det_tb.v
ign run vvp mealy_sim.vvp
ign run gtkwave mealy_seq_det_tb.vcd
```

### CRUCIBLE EXECUTION OUTPUT

```
C:\Users\ignyt\Desktop\ignytion_demo_experiments\04_mealy_seq_det >> ign run iverilog -o mealy_seq_det.vvp -s mealy_seq_det_tb mealy_seq_det.v mealy_seq_det_tb.v
C:\Users\ignyt\Desktop\ignytion_demo_experiments\04_mealy_seq_det >> ign run vvp mealy_seq_det.vvp
VCD info: dumpfile mealy_seq_det_tb.vcd opened for output.
Time=0 | seq_in=0 | state=xx | det_o=0
Time=5000 | seq_in=0 | state=00 | det_o=0
Time=20000 | seq_in=1 | state=00 | det_o=0
Time=25000 | seq_in=1 | state=01 | det_o=0
Time=30000 | seq_in=0 | state=01 | det_o=0
Time=35000 | seq_in=0 | state=10 | det_o=0
Time=40000 | seq_in=1 | state=10 | det_o=1
Time=45000 | seq_in=1 | state=01 | det_o=0
Time=60000 | seq_in=0 | state=01 | det_o=0
Time=65000 | seq_in=0 | state=10 | det_o=0
Time=70000 | seq_in=1 | state=10 | det_o=1
Time=75000 | seq_in=1 | state=01 | det_o=0
Time=80000 | seq_in=0 | state=01 | det_o=0
Time=85000 | seq_in=0 | state=10 | det_o=0
Time=90000 | seq_in=1 | state=10 | det_o=1
Time=95000 | seq_in=1 | state=01 | det_o=0
Time=110000 | seq_in=0 | state=01 | det_o=0
Time=115000 | seq_in=0 | state=10 | det_o=0
Time=125000 | seq_in=0 | state=00 | det_o=0
Time=130000 | seq_in=1 | state=00 | det_o=0
Time=135000 | seq_in=1 | state=01 | det_o=0
Time=140000 | seq_in=0 | state=01 | det_o=0
Time=145000 | seq_in=0 | state=10 | det_o=0
Time=150000 | seq_in=1 | state=10 | det_o=1
Time=155000 | seq_in=1 | state=01 | det_o=0
Time=160000 | seq_in=0 | state=01 | det_o=0
Time=165000 | seq_in=0 | state=10 | det_o=0
Time=170000 | seq_in=1 | state=10 | det_o=1
Mealy sequence detector test passed.
mealy_seq_det_tb.v:70: $finish called at 175000 (1ps)
Time=175000 | seq_in=1 | state=01 | det_o=0
C:\Users\ignyt\Desktop\ignytion_demo_experiments\04_mealy_seq_det >> ign run gtkwave mealy_seq_det_tb.vcd
```

Timing diagram showing the behavior of the 2-bit counter circuit over 80 ns. The signals are:

- clock**: Periodic square wave.
- reset**: Active-low signal, going low at 20 ns and 60 ns.
- seq\_in**: Square wave input.
- present\_state[1:0]**: Current state of the counter. Values shown: 00, 01, 10, 01, 10, 01, 10, 00.
- next\_state[1:0]**: Next state of the counter. Values shown: 00, 01, 10, 00, 01, 10, 00, 01.
- det\_o**: Output signal, which is high when the state is 00 or 10.

det\_o asserts on every completion of **101**, including overlapping occurrences, matching the reference model on all 16 bits of the test pattern. **Mealy sequence detector test passed.**

## EXPERIMENT 05

# Three-Number Adder

Combinational Logic

testbench · PASS/FAIL

## AIM

Design and verify a combinational adder that sums three 8-bit operands into a 10-bit result with no overflow.

## THEORY

Three 8-bit operands (max 255 each) yield at most  $255 \times 3 = 765$ , which requires 10 bits ( $2^{10} = 1024$ ). The design zero-extends each operand to 10 bits before summing, guaranteeing the result never wraps. The logic is purely combinational — no clocking required.

## PORTS

| PORT                   | DIR | DESCRIPTION            |
|------------------------|-----|------------------------|
| a[7:0], b[7:0], c[7:0] | in  | Three 8-bit operands   |
| sum[9:0]               | out | 10-bit sum (a + b + c) |

## RTL DESIGN · THREE\_NUM\_ADDER.V

```

module three_num_adder (
    input wire [7:0] a,
    input wire [7:0] b,
    input wire [7:0] c,
    output wire [9:0] sum
);
    assign sum = {2'b00, a} + {2'b00, b} + {2'b00, c};
endmodule

```

## EXPERIMENT 05 · THREE-NUMBER ADDER — EXECUTION & RESULTS

### TEST STRUCTURE

```
task check_sum(input [7:0] a_value, input [7:0] b_value, input [7:0] c_value);
    a = a_value; b = b_value; c = c_value; #10;
    expected_sum = a_value + b_value + c_value;
    if (sum !== expected_sum[9:0]) begin
        $display("ERROR: a=%0d b=%0d c=%0d expected=%0d got=%0d",
            a_value, b_value, c_value, expected_sum, sum);
    $finish;
    end
endtask

// Test stimulus: boundary cases + parametric sweep
check_sum(8'd0, 8'd0, 8'd0); // Min
check_sum(8'd255, 8'd255, 8'd255); // Max (765)
check_sum(8'd85, 8'd42, 8'd21); // Random values
for (i = 0; i < 8; i = i + 1)
    check_sum(i, i * 3, i * 5);
```

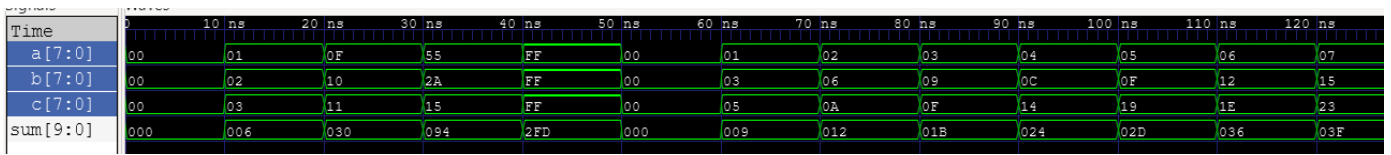
### PROCEDURE

```
cd 05_three_num_adder
ign run iverilog -o three_num_adder.vvp -s three_num_adder_tb three_num_adder.v three_num_adder_tb.v
ign run vvp three_num_adder.vvp
ign run gtkwave three_num_adder_tb.vcd
```

### CRUCIBLE EXECUTION OUTPUT

```
C:\Users\ignyt\Desktop\ignition_demo_experiments\05_three_num_adder >> ign run iverilog -o three_num_adder.vvp -s three_num_adder_tb three_num_adder.v three
e_num_adder_tb.v
C:\Users\ignyt\Desktop\ignition_demo_experiments\05_three_num_adder >> ign run vvp three_num_adder.vvp
VCD info: dumpfile three_num_adder_tb.vcd opened for output.
Time=0 | a=0 | b=0 | c=0 | sum=0
Time=10000 | a=1 | b=2 | c=3 | sum=6
Time=20000 | a=15 | b=16 | c=17 | sum=48
Time=30000 | a=85 | b=42 | c=21 | sum=148
Time=40000 | a=255 | b=255 | c=255 | sum=765
Time=50000 | a=0 | b=0 | c=0 | sum=0
Time=60000 | a=1 | b=3 | c=5 | sum=9
Time=70000 | a=2 | b=6 | c=10 | sum=18
Time=80000 | a=3 | b=9 | c=15 | sum=27
Time=90000 | a=4 | b=12 | c=20 | sum=36
Time=100000 | a=5 | b=15 | c=25 | sum=45
Time=110000 | a=6 | b=18 | c=30 | sum=54
Time=120000 | a=7 | b=21 | c=35 | sum=63
Three-number adder test passed.
three_num_adder_tb.v:55: $finish called at 130000 (1ps)
C:\Users\ignyt\Desktop\ignition_demo_experiments\05_three_num_adder >> ign run gtkwave three_num_adder_tb.vcd
```

## GTKWAVE — WAVEFORM OUTPUT



## RESULT

Sum is correct across all vectors including the maximum  $255+255+255 = 765$ , confirming the 10-bit output never overflows. **Three-number adder test passed.**

## SECTION 3

## Summary of Results

| #  | EXPERIMENT              | CATEGORY       | PASS MESSAGE                         |
|----|-------------------------|----------------|--------------------------------------|
| 01 | D Flip-Flop             | Sequential     | <i>monitor — behaviour verified</i>  |
| 02 | Controlled Counter      | Sequential FSM | Controlled counter test passed.      |
| 03 | Seven-Segment Decoder   | Combinational  | Seven-segment decoder test passed.   |
| 04 | Mealy Sequence Detector | Sequential FSM | Mealy sequence detector test passed. |
| 05 | Three-Number Adder      | Combinational  | Three-number adder test passed.      |

All five experiments compile and simulate cleanly on the Ignition platform. The four Verilog testbenches report PASS; the D lab demonstrates the Ign CLI's end-to-end EDA workflow — **provision** → **run** → **verify** → **reproduce** — across Flip-Flop is verified via simulation log inspection. The both combinational and sequential design classes.

Because the toolset was locked with `ign workspace lock`, the entire lab is reproducible on any machine: `git clone <repo> && ign workspace sync` restores the identical Icarus Verilog and GTKWave versions with no manual installation.